```
        return InMemoryUserRepository.getInstance();
    }
}
```

This offers a static method *get()*, which the consumers can invoke to get hold of an instance of a provider that implements the *UserRepository* interface. What else does the *Application* want?

```
UserRepository repo = UserRepositoryFactory.get();
```

Now the *Application* has ZERO references to the *InMemoryUserRepository*. In other words, the consumer and the provider are no longer tightly coupled. A factory always takes the responsibility of supplying providers to the consumers.

However, a system may have several contracts with several providers. Should we have different factories for different contracts, since the return type of the *get()* method is specific to a given interface?

Not really! With a small tweak, a single factory can supply objects of different interfaces.

```
public static Object get(String key) throws Exception {
    if (key == "user.repository")
      return InMemoryUserRepository.getInstance();
    throw new Exception();
}
```

In the above implementation, the *get()* method is declared to return an *Object*, instead of just a *UserRepository*. You know that *java.lang.Object* is a superclass of everything in Java. In other words, the *get()* method is now equipped to return any object of any class. But how does it choose the object to be returned? This depends on the input from the consumer.

The onus is now on the consumer to specify its preference and also to cast the returned object to the interface it is referring to.

```
UserRepository repo = (UserRepository) ObjectFactory.
get("user.repository");
```

The design looks much better now with interfaces, implementations, singletons and factories.

Framework

There is always scope for improvement, provided that time and other resources are available. Let's look at one such avenue for improvement.

Though we made the *Application* and *InMemoryUserRepository* loosely coupled, the *ObjectFactory* is itself tightly coupled with the providers. Not a good sign, isn't it?

At least in Java, we can overcome that problem by using *reflection*. Look at the following snippet of *get()* method of *ObjectFactory* that uses reflection.

```
public static Object get(String key) throws Exception {
    Properties props = new Properties();
    props.load(new FileReader("config.properties"));
    String className = props.getProperty(key);
    Class<?> claz = Class.forName(className);

    try {
      return claz.newInstance();
    } catch (IllegalAccessException | InstantiationException
e) {
      return claz.getMethod("getInstance").invoke(claz);
    }
}
```

The implementation makes a few assumptions:

1. There will be a configuration file named 'config.properties' which supplies the class names against keys.
2. The classes that are referred to in the configuration file are available on the classpath for loading.
3. The classes either have a publicly accessible default constructor or a statically available *getInstance()* method to get an instance of the class.

The content of the *config.properties* looks like the following:

```
user.repository=com.glarimy.ums.InMemoryUserRepository
```

As long as these expectations are met, the *ObjectFactory* nicely supplies the objects of any class. Suddenly, the *ObjectFactory* looks like a class that is useful in every application, not just for UMS, as it does not have any reference to anything in the UMS. If so, why don't we move it out of the *com.glarimy.ums* package? What is the apt name for a unit that contains such classes with universal appeal? Framework! We better move the *ObjectFactory* to a new package named *com.glarimy.framework*.

Readability and annotations

Frameworks usually make several assumptions and expect developers to build the rest of the system while satisfying those assumptions. Having a static *getInstance()* method on a singleton is such an assumption. But, frankly speaking, the method name can be anything else. Insisting on a certain name to a method is a bit crude. To give freedom to the developers in naming the methods as per their preference, we can improve the framework a bit more.

How about the following code, with *@Singleton* annotation that clearly marks the *InMemoryUserRepository*